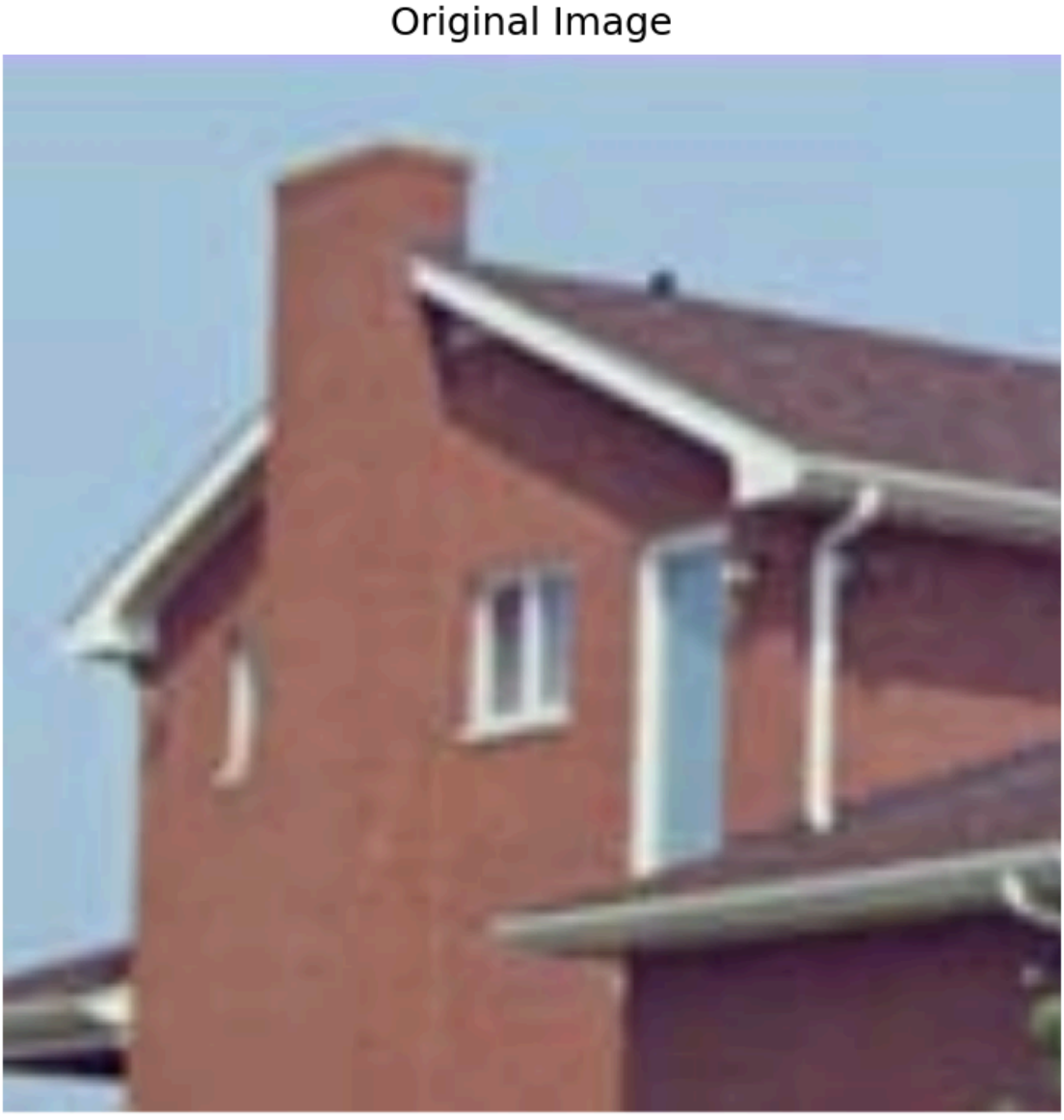
Abstract:

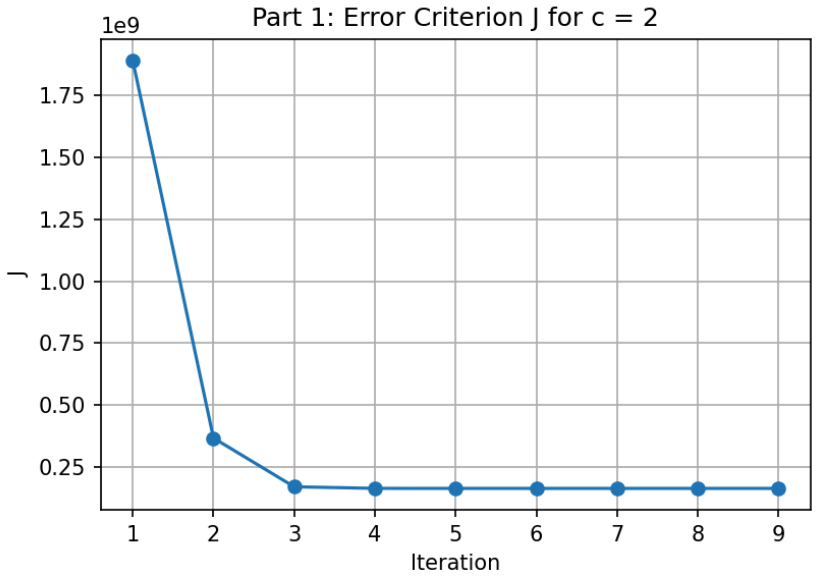
This project uses k-means clustering for image segmentation in RGB space. Each pixel is treated as a vector and grouped with similar pixels based on distance from cluster centers. The cluster means are updated iteratively until the algorithm converges. The segmented results and performance plots show how different cluster choices affect the separation of image regions such as the house and the background.



Results:

Based on the above image of a house, the # of c (2 for part 1 and 5 for part 2) clusters originally start from a random mean RBG vector where each is a value between a range of 0 to 255. From there the euclidean distance are calculated for each pixel in the house relative to each mean for the two clusters, and based on which is a smaller number are assigned to that cluster. Once all pixels have been assigned, the mean of the cluster is calculated based on the pixels belonging to that cluster. With these new means for each cluster, the model iterates through all of the pixels of the house and reassigns pixels based on euclid distance. As you can see after the 2nd iteration the graph converges because the means are roughly accurate for each respective cluster. The error is defined as the formula below (euclidean distance):

$$J = \sum_{j=1}^{c} \sum_{k=1}^{n_j} \|\mathbf{x}_k - \boldsymbol{\mu}_j\|^2 .$$



The results below demonstrate the adjustment of means (**Note:** The vector is 3 variables since each is a value representing red, blue and green which combined can create any colour). The Xie Beni index essentially describes how good the overall clustering is by describing two components; 1.) How compact the cluster are to the mean 2.) how far apart the means for each cluster are from each other. Ideally we want compact clusters with the means/ clusters far apart to really see the groups. Therefore smaller XB index, means better clustering, the formula is shown below:

$$\mathrm{XB}(c) = \frac{\sum_{k=1}^{N} \sum_{j=1}^{c} u_{jk} \left\| \mathbf{x}_k - \boldsymbol{\mu}_j \right\|}{N \min \left\| \boldsymbol{\mu}_i - \boldsymbol{\mu}_j \right\|}$$

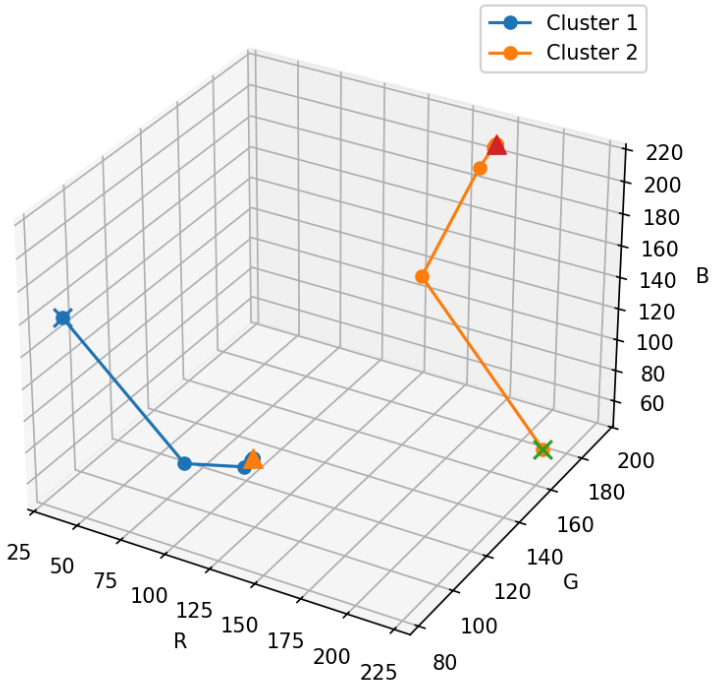
where $u_{jk} = p(w_j|\boldsymbol{x}_k)$ is 1 if $\boldsymbol{x}_k \in j$-th cluster or 0 if $\boldsymbol{x}_k \notin j$-th cluster. Note, $\boldsymbol{\mu}_i$ is the mean value of any classes other than $\boldsymbol{\mu}_j$.

```
PS C:\Users\berha\OneDrive\Documents\ELE888> py kmeans.py
=== Part 1: c = 2 ===
Initial means:
[[ 35.  85. 160.]
Initial means:
[[ 35.  85. 160.]
[[ 35.  85. 160.]
 [220. 170.  55.]]
Final means:
Final means:
[[129.71321666  94.67062582  95.18449033]
 [170.00580105 192.99966578 211.6380434 ]]
Final J: 165177143.79655164

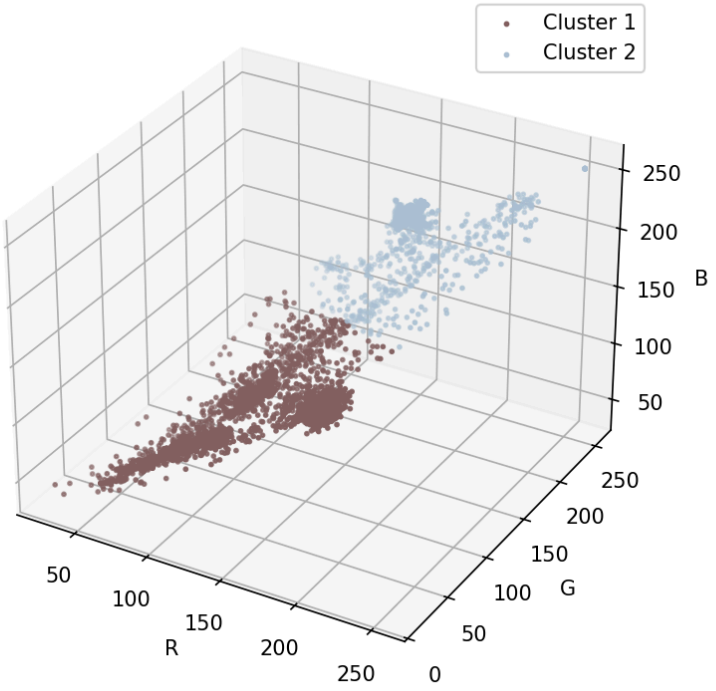
=== Part 2: c = 5, Run 1 ===
Initial means:
[[ 20.  35.  60.]
 [ 70. 110. 170.]
 [130.  95.  65.]
 [185. 150.  90.]
 [245. 225. 185.]]
Final means:
[[ 86.18241718  56.52089561  70.43206638]
 [152.27371736 154.00524866 158.78834799]
 [116.03884366  97.71408859 103.89581896]
 [156.26839892 108.66193775  99.7682023 ]
 [171.59146061 196.96430212 217.36581543]]
Final J: 40805917.241557956
XB index: 0.21412113885864
```

```
=== Part 2: c = 5, Run 2 ===
Initial means:
[[  0.  20.  40.]
 [255.  90.  90.]
 [ 90. 255. 120.]
 [120. 130. 255.]
 [255. 255. 240.]]
Final means:
[[ 86.44458499  56.77069028  70.60643296]
 [155.59055334 111.7605657  103.76177399]
 [116.07923985  98.71405049 105.31572448]
 [166.10282417 191.57948259 212.32815217]
 [222.88671746 223.31798609 220.92613448]]
Final J: 47634045.99111679
XB index: 0.25305568357737057
```

Part 1: Mean Evolution in RGB Space (c = 2)



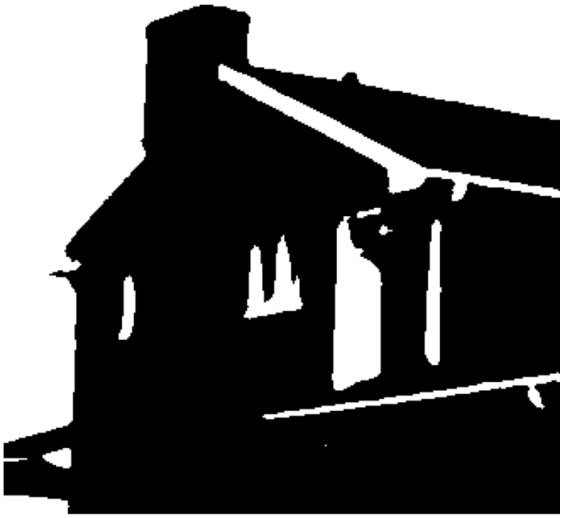
Part 1: Labeled RGB Space (c = 2)

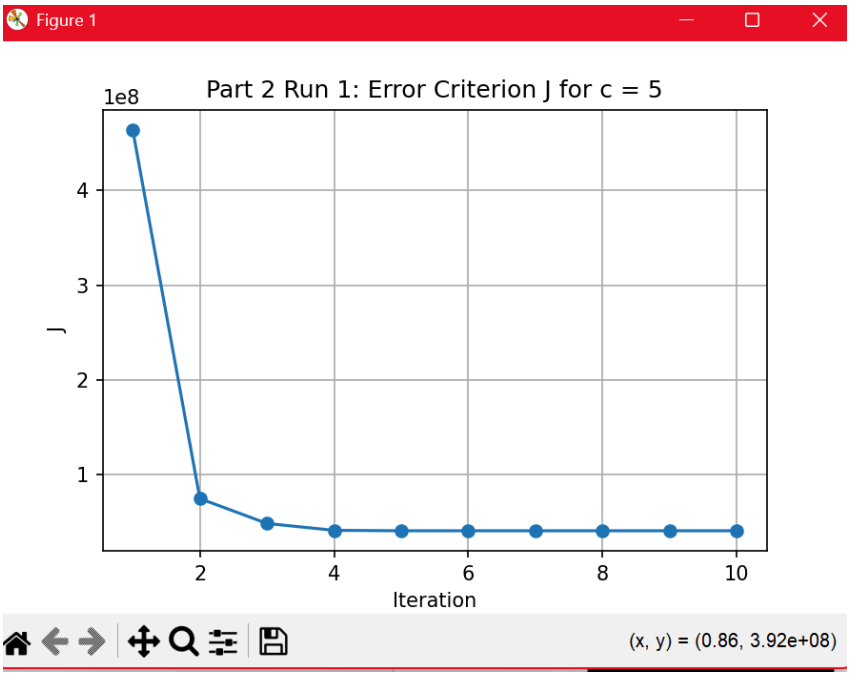


Part 1: Labeled Image Using Mean Colors (c = 2)

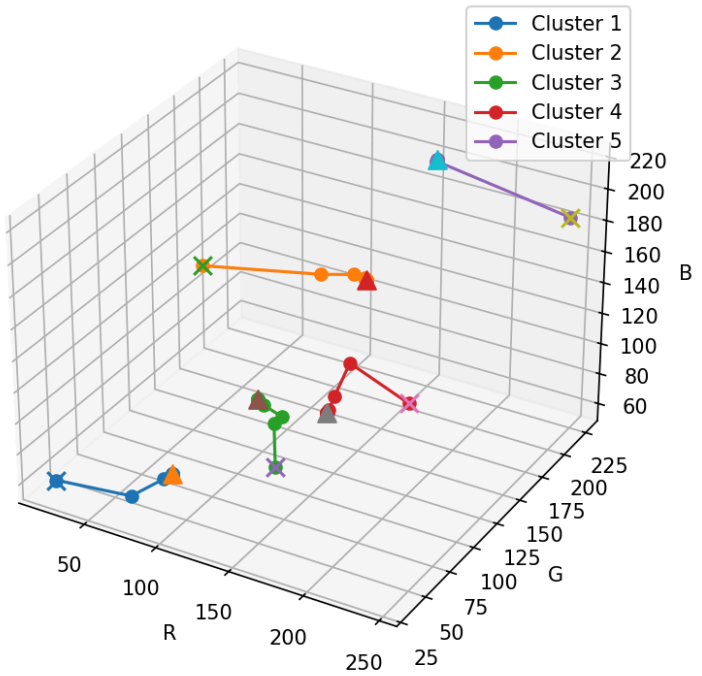


Part 1: Labeled Image Using Artificial Colors (c = 2)

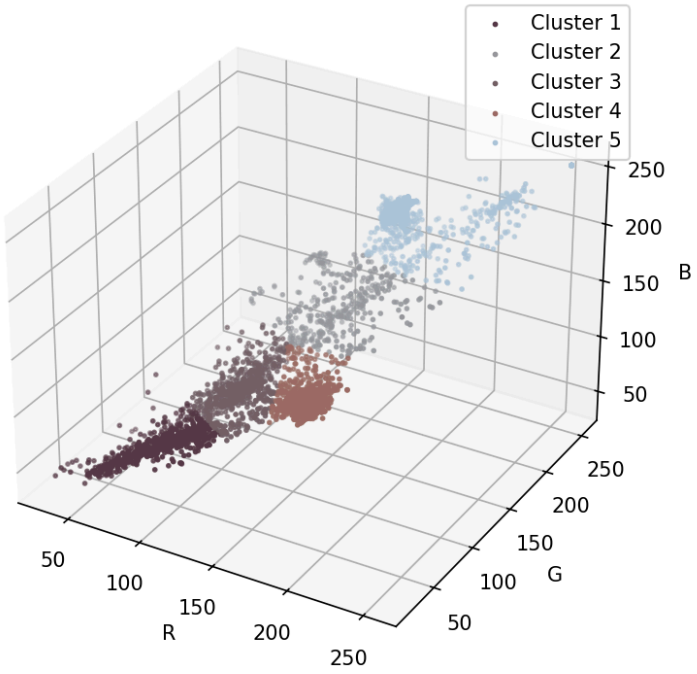




Part 2 Run 1: Mean Evolution in RGB Space (c = 5)



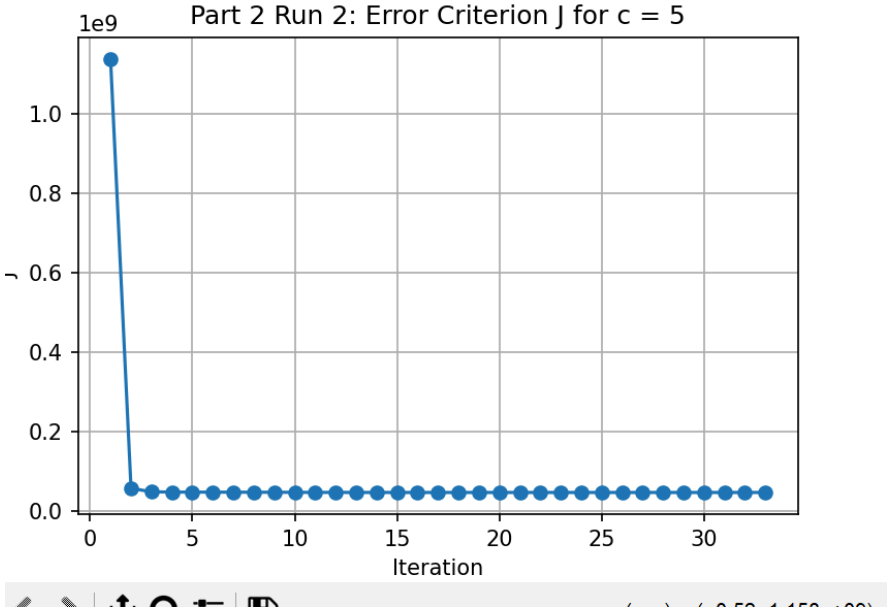
Part 2 Run 1: Labeled RGB Space (c = 5)



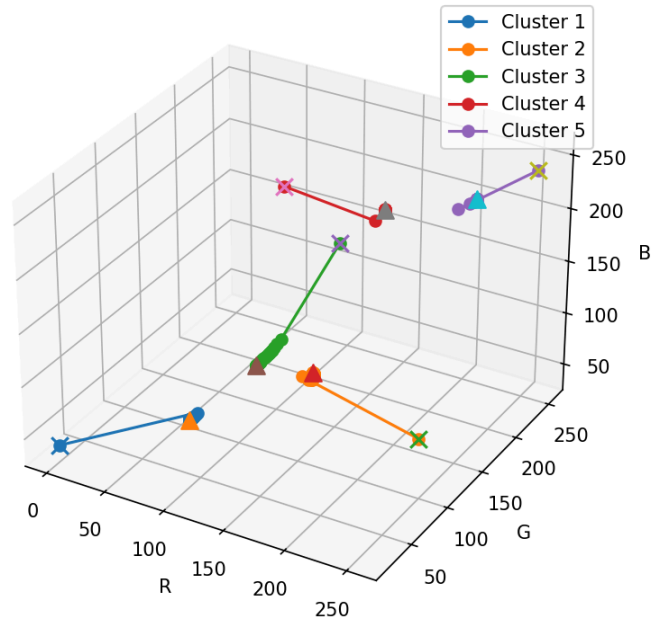
Part 2 Run 1: Labeled Image Using Mean Colors (c = 5)



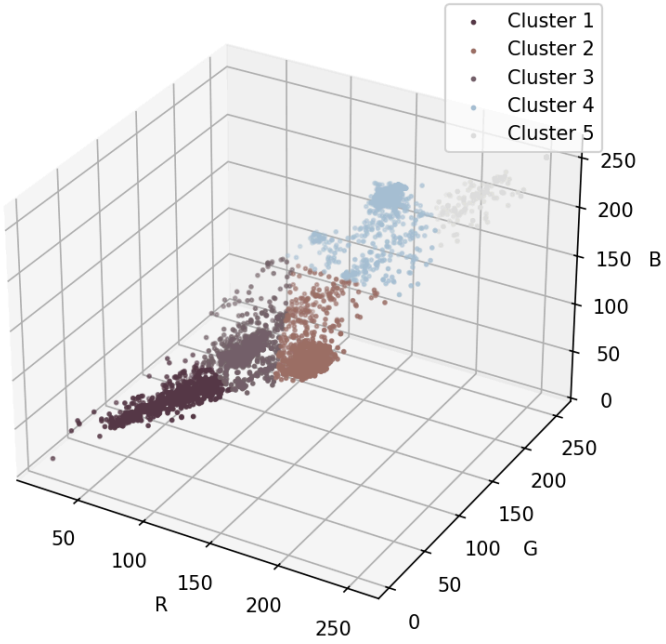
Part 2 Run 2: Error Criterion J for c = 5



Part 2 Run 2: Mean Evolution in RGB Space (c = 5)



Part 2 Run 2: Labeled RGB Space (c = 5)



Part 2 Run 2: Labeled Image Using Mean Colors (c = 5)



Conclusion:

In conclusion, this project showed that k-means clustering can be used effectively for image segmentation by grouping pixels based on their RGB values. Each pixel was treated as a vector, assigned to the nearest cluster mean, and the cluster means were updated iteratively until convergence. The plots of the error criterion and the movement of the cluster centers helped show how the algorithm gradually stabilized.

The results also showed that increasing the number of clusters allows the house to be described with more detail and more colors. With fewer clusters, the image is segmented only into broad regions such as the house and the background. As the number of clusters increases, more parts of the house such as walls, roof areas, shadows, and brighter regions can be represented separately, producing a more detailed description of the image. However, too many clusters may begin to split the image into unnecessary small regions. Overall, the project demonstrated how clustering can simplify and analyze an image, while also showing the tradeoff between simple segmentation and detailed color representation.

Appendix:

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import imageio.v2 as imageio
from mpl_toolkits.mplot3d import Axes3D  # noqa: F401


def load_image(filename):
    I = imageio.imread(filename)

    if I.ndim == 2:
        I = np.stack([I, I, I], axis=-1)

    if I.ndim != 3:
        raise ValueError("Expected a color image.")

    if I.shape[2] == 4:
        I = I[:, :, :3]

    if I.shape[2] != 3:
        raise ValueError(f"Expected 3 channels, but got shape {I.shape}.")

    return I.astype(np.float64)


def pixels_from_image(I):
    M, N, C = I.shape
    X = I.reshape(M * N, C).astype(np.float64)
    return X, M, N


def compute_distances(X, means):
    return np.sum((X[:, None, :] - means[None, :, :]) ** 2, axis=2)


def assign_clusters(X, means):
    dists = compute_distances(X, means)
    return np.argmin(dists, axis=1)
```

```
def recompute_means(X, labels, c, old_means):
    new_means = np.zeros_like(old_means, dtype=np.float64)

    for j in range(c):
        cluster_points = X[labels == j]
        if len(cluster_points) == 0:
            new_means[j] = old_means[j]
        else:
            new_means[j] = np.mean(cluster_points, axis=0)

    return new_means


def compute_J(X, labels, means):
    J = 0.0
    for j in range(len(means)):
        cluster_points = X[labels == j]
        if len(cluster_points) > 0:
            J += np.sum((cluster_points - means[j]) ** 2)
    return J


def xie_beni_index(X, labels, means):
    N = X.shape[0]

    numerator = 0.0
    for j in range(len(means)):
        cluster_points = X[labels == j]
        if len(cluster_points) > 0:
            numerator += np.sum((cluster_points - means[j]) ** 2)

    min_sep = np.inf
    c = len(means)
    for i in range(c):
        for j in range(i + 1, c):
            sep = np.sum((means[i] - means[j]) ** 2)
            if sep < min_sep:
                min_sep = sep
```

```
    if min_sep == 0:
        return np.inf

    return numerator / (N * min_sep)


def kmeans(X, initial_means, max_iter=100, tol=1e-4,
save_mean_history=True):
    means = initial_means.astype(np.float64).copy()
    c = means.shape[0]

    J_history = []
    mean_history = [means.copy()] if save_mean_history else []

    for _ in range(max_iter):
        labels = assign_clusters(X, means)
        J = compute_J(X, labels, means)
        J_history.append(J)

        new_means = recompute_means(X, labels, c, means)
        mean_shift = np.sum(np.linalg.norm(new_means - means, axis=1))

        means = new_means
        if save_mean_history:
            mean_history.append(means.copy())

        if mean_shift < tol:
            break

    labels = assign_clusters(X, means)
    return labels, means, J_history, mean_history


def plot_original_image(I, title="Original Image"):
    plt.figure(figsize=(6, 6))
    plt.imshow(I.astype(np.uint8))
    plt.title(title)
    plt.axis("off")
    plt.show()
```

```
def plot_J_history(J_history, title="K-means Error Criterion J"):
    plt.figure(figsize=(6, 4))
    plt.plot(range(1, len(J_history) + 1), J_history, marker='o')
    plt.xlabel("Iteration")
    plt.ylabel("J")
    plt.title(title)
    plt.grid(True)
    plt.show()


def plot_mean_evolution(mean_history, title="Cluster Mean Evolution"):
    fig = plt.figure(figsize=(7, 6))
    ax = fig.add_subplot(111, projection='3d')

    c = mean_history[0].shape[0]

    for j in range(c):
        traj = np.array([mh[j] for mh in mean_history])
        ax.plot(traj[:, 0], traj[:, 1], traj[:, 2], marker='o',
label=f'Cluster {j+1}')
        ax.scatter(traj[0, 0], traj[0, 1], traj[0, 2], s=80, marker='x')
        ax.scatter(traj[-1, 0], traj[-1, 1], traj[-1, 2], s=80,
marker='^')

    ax.set_xlabel("R")
    ax.set_ylabel("G")
    ax.set_zlabel("B")
    ax.set_title(title)
    ax.legend()
    plt.show()


def plot_labeled_rgb_space(X, labels, means, sample_size=5000,
title="Labeled RGB Space"):
    N = X.shape[0]

    if N > sample_size:
        idx = np.random.choice(N, sample_size, replace=False)
        X_plot = X[idx]
```

```
        labels_plot = labels[idx]
    else:
        X_plot = X
        labels_plot = labels

    colors = means / 255.0

    fig = plt.figure(figsize=(7, 6))
    ax = fig.add_subplot(111, projection='3d')

    for j in range(len(means)):
        pts = X_plot[labels_plot == j]
        if len(pts) > 0:
            ax.scatter(pts[:, 0], pts[:, 1], pts[:, 2], s=3,
c=[colors[j]], label=f'Cluster {j+1}')

    ax.set_xlabel("R")
    ax.set_ylabel("G")
    ax.set_zlabel("B")
    ax.set_title(title)
    ax.legend()
    plt.show()


def reconstruct_image_from_means(labels, means, M, N):
    X_labeled = means[labels]
    I_labeled = X_labeled.reshape(M, N, 3)
    return np.clip(I_labeled, 0, 255).astype(np.uint8)


def reconstruct_image_artificial(labels, M, N, c):
    palette = np.array([
        [0, 0, 0],
        [255, 255, 255],
        [255, 0, 0],
        [0, 255, 0],
        [0, 0, 255],
        [255, 255, 0],
        [255, 0, 255],
        [0, 255, 255]
```

```
    ], dtype=np.uint8)

    if c > len(palette):
        raise ValueError("Need a larger artificial color palette.")

    X_art = palette[labels]
    return X_art.reshape(M, N, 3)


def plot_image(I, title="Image"):
    plt.figure(figsize=(6, 6))
    plt.imshow(I)
    plt.title(title)
    plt.axis("off")
    plt.show()


def main():
    filename = "house.png"
    I = load_image(filename)
    X, M, N = pixels_from_image(I)

    plot_original_image(I, "Original Image")

    # c = 2, different initial means
    init_means_c2 = np.array([
        [35, 85, 160],
        [220, 170, 55]
    ], dtype=np.float64)

    labels2, means2, J2, mean_hist2 = kmeans(
        X,
        initial_means=init_means_c2,
        max_iter=100,
        tol=1e-3,
        save_mean_history=True
    )

    print("=== Part 1: c = 2 ===")
    print("Initial means:")
```

```
    print(init_means_c2)
    print("Final means:")
    print(means2)
    print("Final J:", J2[-1])

    plot_J_history(J2, "Part 1: Error Criterion J for c = 2")
    plot_mean_evolution(mean_hist2, "Part 1: Mean Evolution in RGB Space
(c = 2)")
    plot_labeled_rgb_space(X, labels2, means2, sample_size=5000,
title="Part 1: Labeled RGB Space (c = 2)")

    I_mean2 = reconstruct_image_from_means(labels2, means2, M, N)
    plot_image(I_mean2, "Part 1: Labeled Image Using Mean Colors (c = 2)")

    I_art2 = reconstruct_image_artificial(labels2, M, N, c=2)
    plot_image(I_art2, "Part 1: Labeled Image Using Artificial Colors (c =
2)")

    # c = 5, run 1, different initial means
    init_means_c5_run1 = np.array([
        [20, 35, 60],
        [70, 110, 170],
        [130, 95, 65],
        [185, 150, 90],
        [245, 225, 185]
    ], dtype=np.float64)

    labels5_run1, means5_run1, J5_run1, mean_hist5_run1 = kmeans(
        X,
        initial_means=init_means_c5_run1,
        max_iter=100,
        tol=1e-3,
        save_mean_history=True
    )

    xb1 = xie_beni_index(X, labels5_run1, means5_run1)

    print("\n=== Part 2: c = 5, Run 1 ===")
    print("Initial means:")
    print(init_means_c5_run1)
```

```
    print("Final means:")
    print(means5_run1)
    print("Final J:", J5_run1[-1])
    print("XB index:", xb1)

    plot_J_history(J5_run1, "Part 2 Run 1: Error Criterion J for c = 5")
    plot_mean_evolution(mean_hist5_run1, "Part 2 Run 1: Mean Evolution in
RGB Space (c = 5)")
    plot_labeled_rgb_space(X, labels5_run1, means5_run1, sample_size=5000,
title="Part 2 Run 1: Labeled RGB Space (c = 5)")

    I_mean5_run1 = reconstruct_image_from_means(labels5_run1, means5_run1,
M, N)
    plot_image(I_mean5_run1, "Part 2 Run 1: Labeled Image Using Mean
Colors (c = 5)")

    # c = 5, run 2, different initial means
    init_means_c5_run2 = np.array([
        [0, 20, 40],
        [255, 90, 90],
        [90, 255, 120],
        [120, 130, 255],
        [255, 255, 240]
    ], dtype=np.float64)

    labels5_run2, means5_run2, J5_run2, mean_hist5_run2 = kmeans(
        X,
        initial_means=init_means_c5_run2,
        max_iter=100,
        tol=1e-3,
        save_mean_history=True
    )

    xb2 = xie_beni_index(X, labels5_run2, means5_run2)

    print("\n=== Part 2: c = 5, Run 2 ===")
    print("Initial means:")
    print(init_means_c5_run2)
    print("Final means:")
    print(means5_run2)
```

```
    print("Final J:", J5_run2[-1])
    print("XB index:", xb2)

    plot_J_history(J5_run2, "Part 2 Run 2: Error Criterion J for c = 5")
    plot_mean_evolution(mean_hist5_run2, "Part 2 Run 2: Mean Evolution in
RGB Space (c = 5)")
    plot_labeled_rgb_space(X, labels5_run2, means5_run2, sample_size=5000,
title="Part 2 Run 2: Labeled RGB Space (c = 5)")

    I_mean5_run2 = reconstruct_image_from_means(labels5_run2, means5_run2,
M, N)
    plot_image(I_mean5_run2, "Part 2 Run 2: Labeled Image Using Mean
Colors (c = 5)")

    print("\n=== Comparison of c = 5 runs ===")
    print(f"Run 1 XB index = {xb1}")
    print(f"Run 2 XB index = {xb2}")

    if xb1 < xb2:
        print("Run 1 is better according to the XB index.")
    elif xb2 < xb1:
        print("Run 2 is better according to the XB index.")
    else:
        print("Both runs are equal according to the XB index.")


if __name__ == "__main__":
    main()
```